

1/16/24, 9:48 PM WorkFlowy - Organize your brain.

Docker-01 task

- Create new custom image from Dockerfile
- Container executed from this images must return date when request come to /date url
- Example if I curl to 172.17.0.2/date container must return date and word (even or odd)
- If date end with odd second return "21:59:25 ODD" if with even number then "21:59:26 EVEN"

- For scripting you can use all languages which you now, also can use chatgpt for that After
- when image is done test it.
- Then you need to up Sonatype nexus application (image sonatype/nexus3) Create docker
- hosted repository on it upload your image to it!
-

01.Create new custom Dockerfile

vim Dockerfile

FROM python:3.14

RUN pip install flask

COPY app.py /app/app.py

WORKDIR /app

EXPOSE 2026

CMD ["python", "app.py"]

```
FROM python:3.14
RUN pip install flask
COPY app.py /app/app.py
WORKDIR /app
EXPOSE 2026
CMD ["python", "app.py"]
```

02. Create script from chatgpt

vim app.py

```
from flask import Flask
```

```
from datetime import datetime
```

```
app = Flask(__name__)
```

```
@app.route("/date")
```

```
def date():
```

```
    now = datetime.now()
```

```
    status = "EVEN" if now.second % 2 == 0 else "ODD"
```

```
    return f"{now.strftime('%H:%M:%S')} {status}"
```

```
app.run(host="0.0.0.0", port=2026)
```

```
from flask import Flask
from datetime import datetime

app = Flask(__name__)

@app.route("/date")
def date():
    now = datetime.now()
    status = "EVEN" if now.second % 2 == 0 else "ODD"
    return f"{now.strftime('%H:%M:%S')} {status}"

app.run(host="0.0.0.0", port=2026)
```

03. Build and check images

docker build -t myapp .

```
[root@rhel9 ~]# docker build -t myapp .
[+] Building 62.9s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 211B
=> [internal] load metadata for docker.io/library/python:3.14
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 371B
=> [1/4] FROM docker.io/library/python:3.14@sha256:5f1cdbcab9a50594a79502dd73e885456d2a2fc31f1a1fa18484815b37ee9152
=> => resolve docker.io/library/python:3.14@sha256:5f1cdbcab9a50594a79502dd73e885456d2a2fc31f1a1fa18484815b37ee9152
=> => sha256:01003eb27664213854421449b5a0e89568412b728881bfca68156aa5348a4a16 29.64MB / 29.64MB
=> => sha256:c4a0714943f9328a891e10a6e8394cd0428564c051892c3e7219d5a983391593 249B / 249B
=> => sha256:b6cb0545f5e8bc37842dda4e1c5e4dcfddac9c74059fe29dd87fd05ee2e61015 6.09MB / 6.09MB
=> => sha256:65956eab0c1ca261c14a3e13055ec8ca4f6ec75198f6ef0704d5194884466582 236.26MB / 236.26MB
=> => sha256:9e002cae30c445bfdf70841ba6a8d772e913914a00fa0e0f7bd0e4c089aaa838 67.79MB / 67.79MB
=> => sha256:b453535073b6238aba838c5d063a1468b4d6a9aa54f81de64a4de304dc456685 25.64MB / 25.64MB
=> => sha256:b890c9407285c31d25426ef154b55c72e225f19b478a59451b01a8a44f5ea4f7 49.31MB / 49.31MB
=> => extracting sha256:b890c9407285c31d25426ef154b55c72e225f19b478a59451b01a8a44f5ea4f7
=> => extracting sha256:b453535073b6238aba838c5d063a1468b4d6a9aa54f81de64a4de304dc456685
=> => extracting sha256:9e002cae30c445bfdf70841ba6a8d772e913914a00fa0e0f7bd0e4c089aaa838
=> => extracting sha256:65956eab0c1ca261c14a3e13055ec8ca4f6ec75198f6ef0704d5194884466582
=> => extracting sha256:b6cb0545f5e8bc37842dda4e1c5e4dcfddac9c74059fe29dd87fd05ee2e61015
=> => extracting sha256:01003eb27664213854421449b5a0e89568412b728881bfca68156aa5348a4a16
=> => extracting sha256:c4a0714943f9328a891e10a6e8394cd0428564c051892c3e7219d5a983391593
=> [2/4] RUN pip install flask
=> [3/4] COPY app.py /app/app.py
=> [4/4] WORKDIR /app
=> => exporting layers
=> => exporting manifest sha256:2b71c4f4168a848072b672da4928d481e0b565a87a45bdc57b560963f4dbfe6f
=> => exporting config sha256:c164f2ce44f69c789818296d20468f3de7304bd2487968486ba692c4017f8ad4
=> => exporting attestation manifest sha256:2f9e5aeb60b1f5eb76626a031f58ac024e012e0ecdff805ecb9bf2c4826e07d30
=> => exporting manifest list sha256:2ed98c00b5a5bf6f0a16be87c1f8961311c736279a1fbc09edc0452e8c6fef4
=> => naming to docker.io/library/myapp:latest
=> => unpacking to docker.io/library/myapp:latest
[root@rhel9 ~]#
```

04. Run docker and check it

docker run -d --name app -p 2026:5000 myapp:latest

```
[root@rhel9 ~]# docker run -d --name app -p 2026:5000 myapp:latest
bcfd424effca63ce807cf239a6fc2047a7df7a96cb08568357c18782e0733fa/
[root@rhel9 ~]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
bcfd424effca   myapp:latest   "python app.py"         5 seconds ago Up 4 seconds   0.0.0.0:2026->5000/tcp, [::]:2026->5000/tcp   app
[root@rhel9 ~]#
```

`curl 172.17.0.2:2026/date`

```
[root@rhel9 ~]# curl 172.17.0.2:2026/date
09:30:27 ODD[ root@rhel9 ~]# curl 172.17.0.2:2026/date
09:30:28 EVEN[ root@rhel9 ~]# curl 172.17.0.2:2026/date
09:30:35 ODD[ root@rhel9 ~]# curl 172.17.0.2:2026/date
09:30:36 EVEN[ root@rhel9 ~]#
```

Create nexus host repository

01. Download from the Nexus repository.

`docker pull sonatype/nexus3`

```
[root@rhel9-2 ~]# docker pull sonatype/nexus3
Using default tag: latest
latest: Pulling from sonatype/nexus3
c8a6d0007565: Pull complete
55afa1ecc21d: Pull complete
0b233a12c837: Pull complete
c366415e651d: Pull complete
7fc83daa7e59: Pull complete
af14289d993b: Pull complete
493934b287a1: Pull complete
87e7b3dfbe70: Pull complete
82ea9ed274e6: Download complete
Digest: sha256:6e8905c14210c595a8177910179ce080cdc22fa3f6b660a13e6dfa1f8378a866
Status: Downloaded newer image for sonatype/nexus3:latest
docker.io/sonatype/nexus3:latest
[root@rhel9-2 ~]#
```

02. Run docker and check it

`docker run -d --name nexus -p 8081:8081 -p 8082:8082 sonatype/nexus3:latest`

`docker update --restart always nexus` (vm rebootdan sonrada Docker container işləsin (qalxsın))

```
[root@rhel9-2 ~]# docker run -d --name nexus -p 8081:8081 -p 8082:8082 sonatype/nexus3:latest
fc7c9f221e9527f4afa6a4bab1094336dec0a80610057dc6f93ca1908a70f2f2
[root@rhel9-2 ~]# docker update --restart always nexus
nexus
[root@rhel9-2 ~]# docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS
fc7c9f221e95   sonatype/nexus3:latest  "/opt/sonatype/nexus..."  15 seconds ago  Up 15 seconds  0.0.0.0:8081-8082->8081-8082/tcp, [::]:8081-8082->8081-8082/tcp
[root@rhel9-2 ~]#
```

03. Go to web interface <http://192.168.153.10:8081/> and change admin password

```
# docker exec nexus cat /nexus-data/admin.password
```

```
[root@rhel9-2 ~]# docker exec -it nexus sh
/opt/sonatype $ cat /
bin/          dev/          home/         media/        nexus-data/  proc/         run/          srv/          tmp/          var/
etc/          lib/          mnt/         opt/         root/        sbin/         sys/         usr/
/opt/sonatype $ cat
nexus/
/opt/sonatype $ cat sonatype-work/
sonatype-work/
/opt/sonatype $ cat sonatype-work/nexus3/
start-nexus-repository-manager.sh
admin.password  clean_cache  downloads/   javaprefs/   log/          tmp/
blobs/          db/          etc/         keystores/   restore-from-backup/
/opt/sonatype $ cat sonatype-work/nexus3/admin.password
04de0852-cb77-4c75-b271-cfceaaff6e2f9/opt/sonatype $
```

← → ↻ ⚠ Not secure 192.168.153.11:8081/#/login

 **sonatype nexus repository**
Community Edition



Welcome

Login to continue

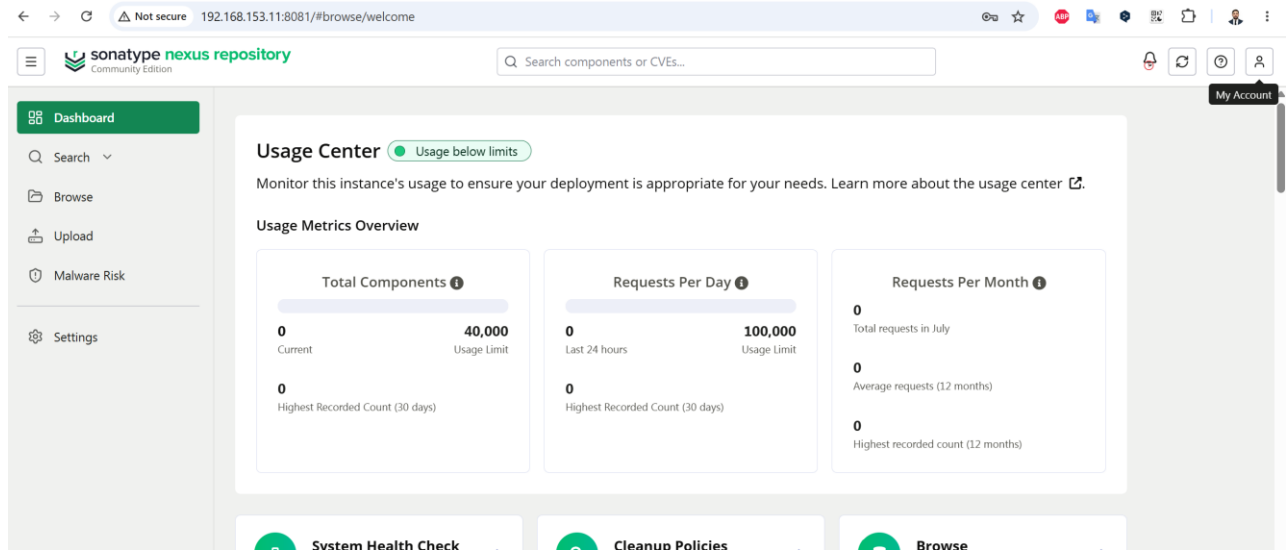
 To log in for the first time, use the generated admin password located at:
`/nexus-data/admin.password`

Username

Password

Log in

04. **Create repositories** Repositories • create repository • select type: Docker (hosted) • name: docker-local • on HTTP port 8082 • change deploy policy: Allow redeploy • click create repository



Docker repository yaradanda (Docker (hosted) secilir)

The screenshot shows the Sonatype Nexus Repository Community Edition 'Repositories' configuration page. The sidebar includes navigation options: Dashboard, Search, Browse, Upload, Malware Risk, and Settings. The main content area is titled 'Repositories' and shows the configuration for a new repository named 'docker-local'. The 'Name' field is set to 'docker-local'. The 'Online' checkbox is checked. The 'Repository Connectors' section includes a description and two options: 'Path based routing' (selected) and 'Other Connectors'. The 'Path based routing' option is selected, and the 'HTTP' checkbox is checked, with the port '8082' entered. The 'HTTPS' checkbox is unchecked. The 'Allow Anonymous Docker Pulls for This Repository' section has an unchecked checkbox.

Repositories / **Select Recipe** / **Create Repository: docker (hosted)**

Name: A unique identifier for this repository
docker-local

Online: ☒ If checked, the repository accepts incoming requests

Repository Connectors

Connectors allow Docker clients to connect directly to hosted registries, but are not always required. Consult our [documentation](#) for which connector is appropriate for your use case. For information on scaling the repositories see our [scaling documentation](#).

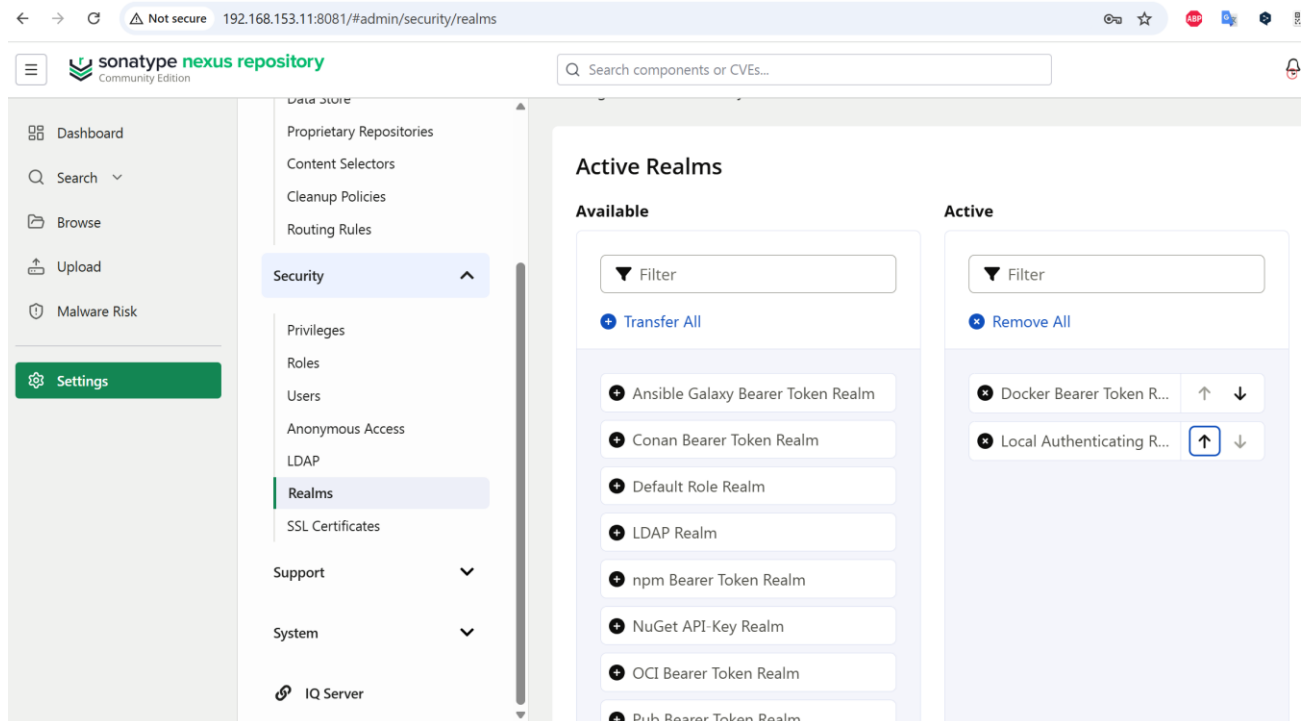
☐ **Path based routing**
Use the following URL with your Docker client to pull images from this repository:
company.com/repo-name/image.tag

☒ **Other Connectors**
Select subdomain, ports, or leave blank if using a reverse proxy.

HTTP:
Create an HTTP connector at specified port. Normally used if the server is behind a secure proxy.
☒ 8082

HTTPS:
Create an HTTPS connector at specified port. Normally used if the server is configured for https.
☐

Allow Anonymous Docker Pulls for This Repository:
☐ Allow anonymous Docker pulls for this repository (Global Anonymous Access and Docker Bearer Token Realm required)



06. Add to insecure registry restart docker and run nexus

vim /etc/docker/daemon.json

```
{  
  "insecure-registries": ["192.168.153.11:8082"]  
}
```

systemctl restart docker

docker start nexus

```
{  
  "insecure-registries": ["192.168.153.11:8082"]  
}
```

07. Check image and tag it

docker images

docker tag dockertask:01 192.168.153.11:8082/dockertask:01

08. Login to NEXUS and push image

docker login 192.168.153.11:8082

docker push 192.168.153.11:8082/dockertask:01

```
[root@rhel9 ~]# vim /etc/docker/daemon.json
[root@rhel9 ~]# systemctl restart docker
[root@rhel9 ~]# docker login 192.168.153.11:8082
Username: admin
Password:

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
[root@rhel9 ~]#
```

```
[root@rhel9 ~]# docker images
IMAGE          ID              DISK USAGE   CONTENT SIZE  EXTRA
myapp:latest   1507dd2505a3    1.62GB       421MB         U
[root@rhel9 ~]# docker tag myapp:latest 192.168.153.11:8082/myapp:1.0
[root@rhel9 ~]# docker images
IMAGE          ID              DISK USAGE   CONTENT SIZE  EXTRA
192.168.153.11:8082/myapp:1.0  1507dd2505a3    1.62GB       421MB         U
myapp:latest   1507dd2505a3    1.62GB       421MB         U
[root@rhel9 ~]#
```

```
[root@rhel9 ~]# docker push 192.168.153.11:8082/myapp:1.0
The push refers to repository [192.168.153.11:8082/myapp]
65956eab0c1c: Pushed
01003eb27664: Pushed
c4a0714943f9: Pushed
b890c9407285: Pushed
b453535073b6: Pushed
b6cb0545f5e8: Pushed
dcdcf354268c0: Pushed
5d923d57002c: Pushed
594849abadae: Pushed
4f4fb700ef54: Pushed
9e002cae30c4: Pushed
1.0: digest: sha256:1507dd2505a3f5829375e1576c759c2dc972c254c78702680dc40d6b70dc30c4 size: 856
[root@rhel9 ~]#
```




🗄 Dashboard

🔍 Search ▾

📁 **Browse**

📤 Upload

🛡 Malware Risk

⚙ Settings

🗄 **Browse** / 🗄 docker-local

HTML View

📁 v2
├── 📁 blobs
├── 📁 myapp
│ ├── 📁 manifests
│ └── 📁 tags